Rapport Technique — Phase 3 : Application Mobile CCDS

Projet	Application Citoyenne de Signalement (CCDS)
Phase	3 sur 6 : Développement de l'Application Mobile
Auteur	Manus AI
Date	26 Février 2026
Commit	`39bd5c5`

1. Introduction

Ce document présente le rapport technique détaillé de la **Phase 3** du projet CCDS, consacrée au développement de l'application mobile multiplateforme (iOS et Android). L'objectif de cette phase était de produire une application fonctionnelle, robuste et intuitive permettant aux citoyens de signaler des incidents, de suivre leur résolution et d'interagir avec les services de la commune.

Le développement a été réalisé en **React Native** avec le framework **Expo**, un choix stratégique visant à maximiser la réutilisation du code entre iOS et Android, tout en simplifiant la gestion des dépendances natives et le processus de build.

2. Métriques du Développement

L'effort de développement de cette phase peut être quantifié par les métriques suivantes, collectées directement depuis le code source et l'historique Git.

2.1. Volume de Code

Un total de **17 fichiers** sources ont été créés, représentant **2 317 lignes de code** (incluant la configuration), démontrant la complétude de la base de code pour une première version.

Fichier	Lignes de Code
`src/screens/CreateIncidentScreen.tsx`	361
`src/screens/IncidentDetailScreen.tsx`	323
`src/screens/MapScreen.tsx`	265
`src/components/ui.tsx`	247
`src/screens/MyIncidentsScreen.tsx`	224
`src/services/api.ts`	209
`src/screens/LoginScreen.tsx`	157
`src/screens/RegisterScreen.tsx`	155
`src/navigation/RootNavigator.tsx`	132
`src/services/AuthContext.tsx`	98
Total (Code Source)	**2 171**
Total (Avec Config)	**2 317**

2.2. Commit Git

La totalité de la phase a été livrée en un seul commit atomique, `39bd5c5`, pour garantir la cohérence et la traçabilité. Ce commit a introduit **2 332 nouvelles lignes de code** et en a modifié 15.

3. Architecture Technique

L'architecture de l'application a été conçue pour être modulaire, évolutive et facile à maintenir.

3.1. Structure des Dossiers

Le code est organisé de manière logique pour séparer les préoccupations :

- `/src/components` : Composants d'interface utilisateur réutilisables (`Button` , `Input` , `IncidentCard`).
- `/src/navigation` : Logique de navigation et définition des écrans (`RootNavigator`).
- `/src/screens` : Écrans principaux de l'application, chacun représentant une vue distincte.
- `/src/services` : Logique métier, appels API et gestion de l'état (`api.ts` , `AuthContext.tsx`).

3.2. Gestion de l'État

Pour cette première version, une approche légère et performante a été privilégiée en utilisant les outils natifs de React :

- **React Context (`AuthContext`)** : Pour la gestion de l'état d'authentification global. Il fournit les informations de l'utilisateur connecté et les fonctions `login` , `register` , `logout` à l'ensemble de l'application.
- **Hooks locaux (`useState` , `useEffect`)** : Pour la gestion de l'état local des composants et des écrans, ce qui est suffisant pour les besoins actuels et évite la complexité d'une librairie externe comme Redux.

3.3. Communication API

Le fichier `src/services/api.ts` centralise toute la communication avec le backend PHP. Il assure :

- **Une instance `fetch` configurée** : Pour tous les appels HTTP.

- **Gestion automatique du token JWT** : Le token est récupéré depuis le `SecureStore` et ajouté à l'en-tête `Authorization` de chaque requête authentifiée.
- **Typage fort** : Toutes les réponses et les corps de requête sont typés avec TypeScript (`User` , `Incident` , `Category` , etc.), garantissant la sécurité des données à travers l'application.
- **Endpoints structurés** : Les appels sont organisés par ressource (`authApi` , `incidentsApi` , etc.).

4. Fonctionnalités Clés Implémentées

L'application couvre l'ensemble du parcours utilisateur citoyen.

4.1. Authentification Complète

- **Écrans de Connexion et d'Inscription** : Formulaires avec validation en temps réel et gestion des erreurs.
- **Stockage Sécurisé du Token** : Utilisation de `expo-secure-store` pour persister le token JWT de manière chiffrée sur l'appareil.
- **Navigation Protégée** : Le `RootNavigator` bascule automatiquement entre la stack d'authentification et la stack principale de l'application en fonction de la présence d'un token valide.

4.2. Création de Signalement (Core Loop)

L'écran `CreateIncidentScreen` est le cœur de l'application et intègre plusieurs API natives :

- **`expo-image-picker`** : Permet à l'utilisateur de prendre une photo avec la caméra ou d'en choisir une depuis sa galerie.
- **`expo-location`** : Récupère les coordonnées GPS de l'utilisateur avec une haute précision et effectue un géocodage inverse pour obtenir une adresse postale lisible.
- **Envoi `multipart/form-data`** : Le signalement (données textuelles + image) est envoyé à l'API via une requête `FormData` .

4.3. Visualisation des Données

- **Carte Interactive (`MapScreen`)** : Utilise `react-native-maps` pour afficher tous les signalements. Les marqueurs sont colorés dynamiquement en fonction du statut de l'incident, permettant une visualisation rapide de l'état des lieux dans la commune.
- **Liste des Signalements (`MyIncidentsScreen`)** : Affiche une liste paginée et infinie des signalements de l'utilisateur, avec des filtres par statut et une fonctionnalité "Pull to Refresh".
- **Écran de Détail (`IncidentDetailScreen`)** : Présente une vue complète d'un signalement, incluant une galerie des photos, l'historique des changements de statut et un fil de commentaires publics.

5. Dépendances Majeures

Le projet s'appuie sur un ensemble de bibliothèques robustes et maintenues par la communauté Expo et React Native.

Dépendance	Version	Rôle
`expo`	52.0.0	Framework de base
`react` / `react-native`	18.3.1 / 0.76.5	Librairie UI
`@react-navigation`	6.11.0	Gestion de la navigation
`react-native-maps`	1.18.0	Affichage des cartes interactives
`expo-camera`	16.0.0	Accès à la caméra native
`expo-location`	18.0.0	Géolocalisation et géocodage
`expo-image-picker`	16.0.0	Accès à la galerie de photos
`expo-secure-store`	14.0.0	Stockage chiffré
`typescript`	5.3.3	Sura-ensemble de JavaScript typé

6. Conclusion et Prochaines Étapes

La Phase 3 a abouti à la création d'une application mobile complète, fonctionnelle et prête pour les tests utilisateurs. L'architecture mise en place est solide et permettra d'itérer facilement pour ajouter de nouvelles fonctionnalités.

La prochaine étape, la **Phase 4**, se concentrera sur le développement du **Back-Office Web d'Administration**, qui permettra aux agents municipaux de traiter les signalements reçus via l'application mobile.